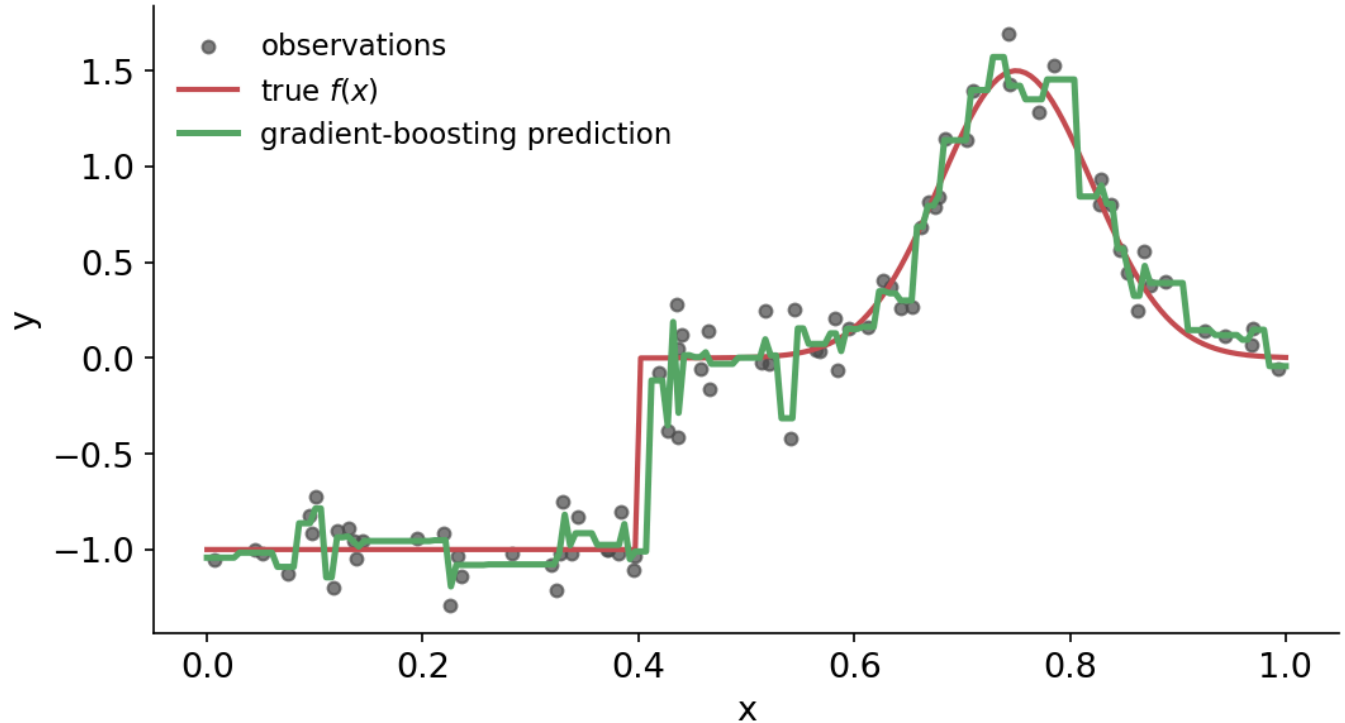
How BART works

Bayesian Additive Regression Trees, from scratch

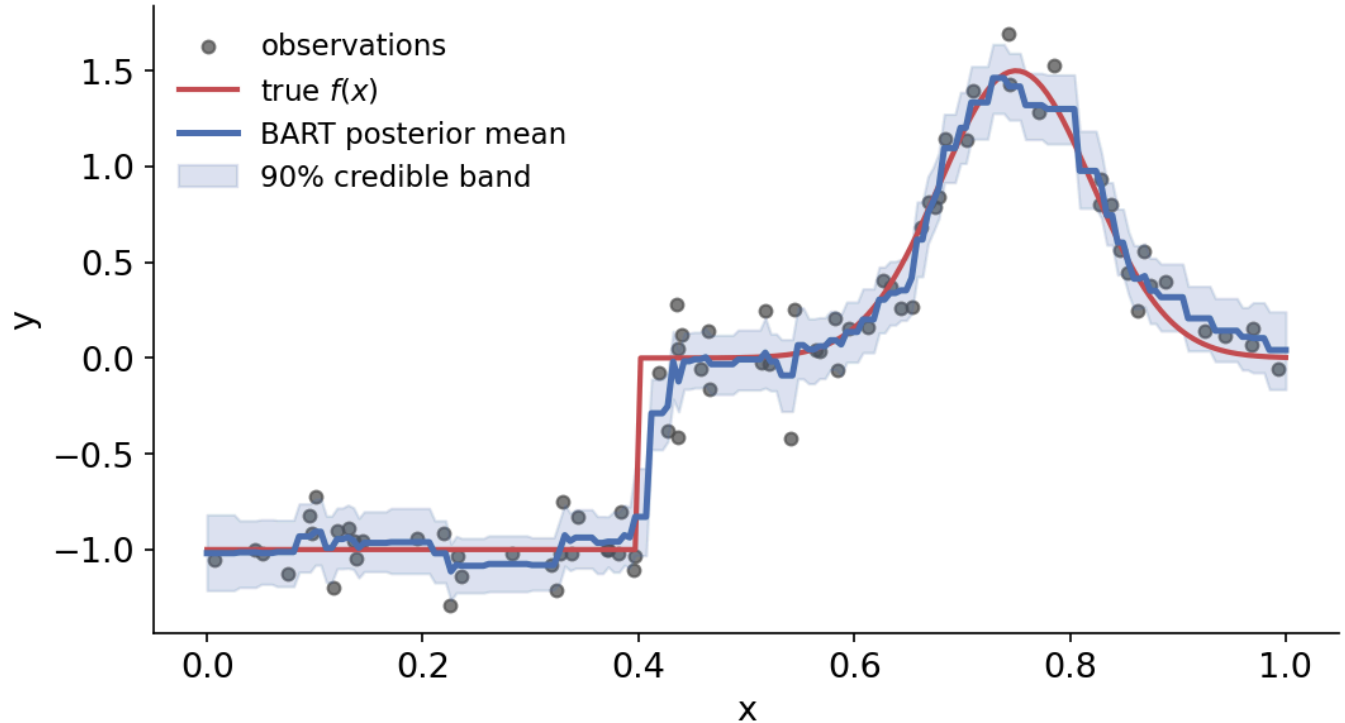
Chris Fonnesbeck · PyData London 2026

Start with a familiar model

Gradient boosting



Same data, with BART



So what is BART?

STRUCTURE	**Like gradient boosting** — a prediction is a sum of many small trees.
FITTING	**Not stagewise boosting** — posterior backfitting, not greedy residual chasing.
OUTPUT	**Bayesian** — posterior draws over functions give credible bands.

Bayesian inference, briefly

Bayes' rule

$$\underbrace{P(\theta \mid D)}_{\text{posterior}} = \frac{\overbrace{P(D \mid \theta)}^{\text{likelihood}} \cdot \overbrace{P(\theta)}^{\text{prior}}}{\underbrace{P(D)}_{\text{evidence}}}$$

The Gaussian model

Priors

Mean:

$$\mu \sim \mathcal{N}(\mu_0, \tau^2)$$

Noise:

$$\sigma^2 \sim \text{Inv-}\chi^2(\nu, \lambda)$$

Likelihood

Given the mean and noise variance:

$$y_i \mid \mu, \sigma^2 \sim \mathcal{N}(\mu, \sigma^2)$$

Bayes combines these into conditional updates for μ and σ^2.

Conditional update for the mean

Given the current σ^2, Normal prior + Normal likelihood gives:

$$\mu \mid y \sim \mathcal{N}\left(\underbrace{\frac{\tau^{-2}\mu_0 + \sigma^{-2} n\,\bar{y}}{\tau^{-2} + \sigma^{-2} n}}_{\text{precision-weighted average}},\ \underbrace{\left(\tau^{-2} + \sigma^{-2} n\right)^{-1}}_{\text{combined precision}} \right)$$

MCMC, briefly

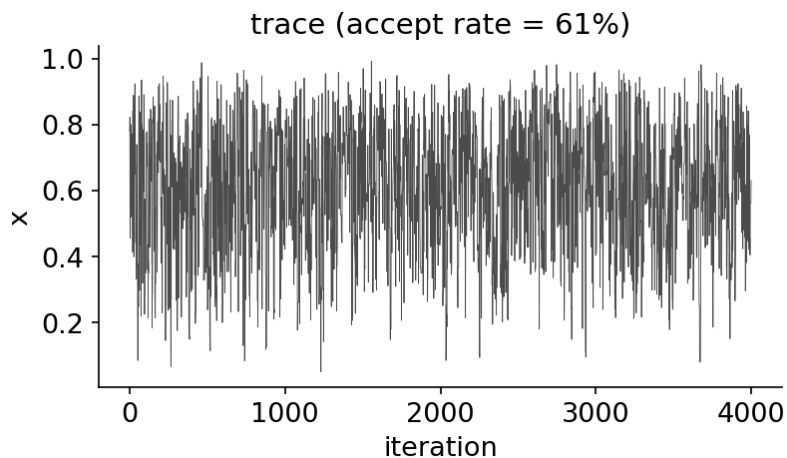
Metropolis–Hastings

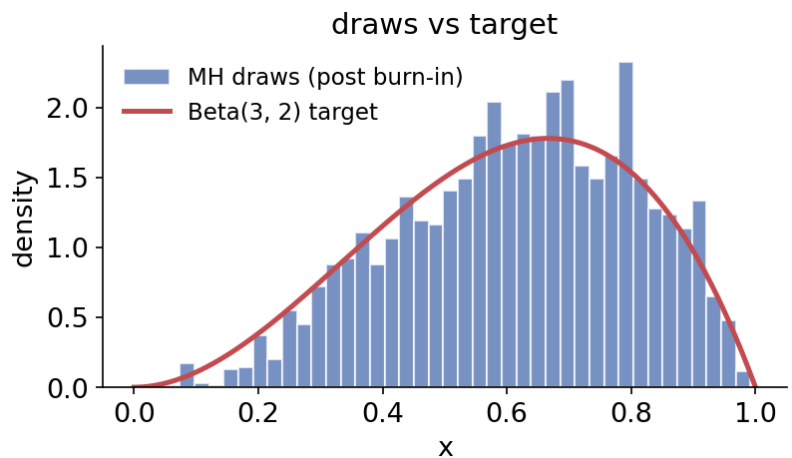
Propose θ', accept with probability

$$\alpha = \min\left(1, \underbrace{\frac{P(\theta' \mid D)}{P(\theta \mid D)}}_{\text{posterior ratio}} \cdot \underbrace{\frac{q(\theta \mid \theta')}{q(\theta' \mid \theta)}}_{\text{proposal correction}}\right)$$

Metropolis–Hastings in seven lines

```python
# random-walk Metropolis on a Beta(3, 2) target
for t in range(n_draws):
    x_new = x + rng.normal(scale=step)
    lp_new = log_target(x_new)
    if np.log(rng.uniform()) < lp_new - lp:   # accept?
        x, lp = x_new, lp_new
    draws[t] = x
```





Partial residuals

Hold the other trees fixed

$$\hat{f}_{-j}(X) = \sum_{k \neq j} f_k(X)$$

Current prediction from every tree except j.

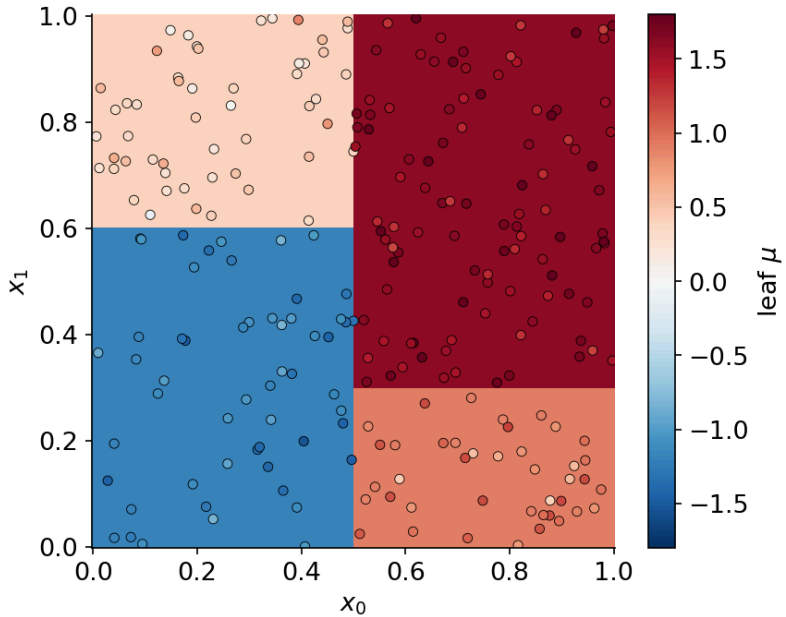
Update tree j on the leftover

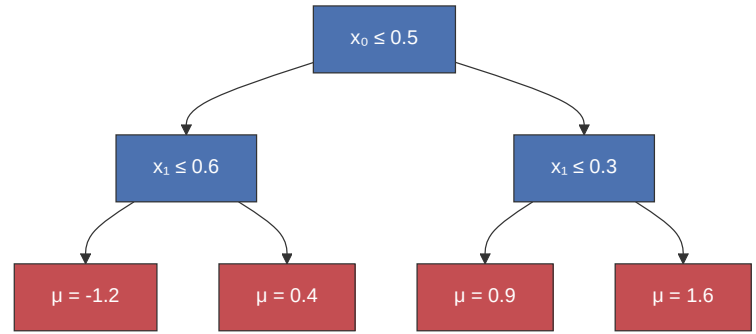
$$r_j = y - \hat{f}_{-j}(X)$$

This temporary target is what tree j sees.

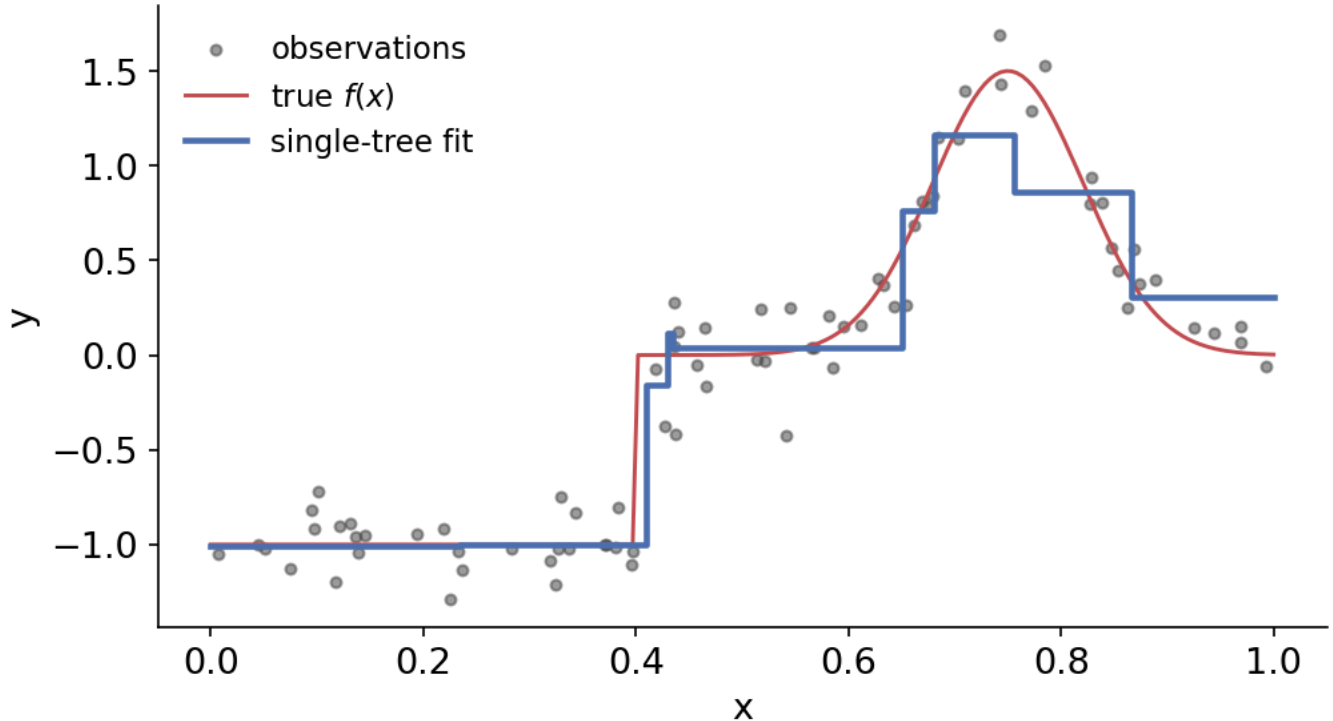
A single regression tree

A tree partitions the input space



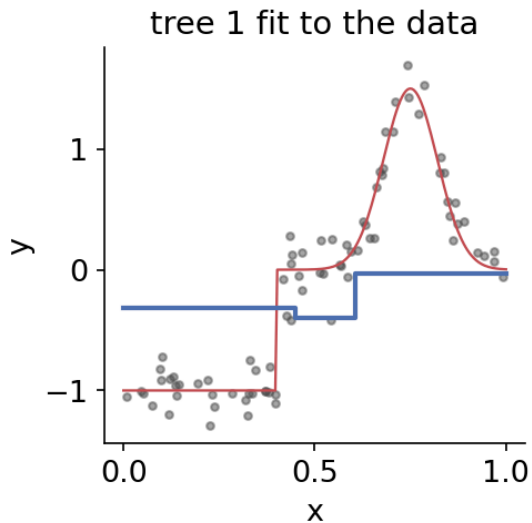


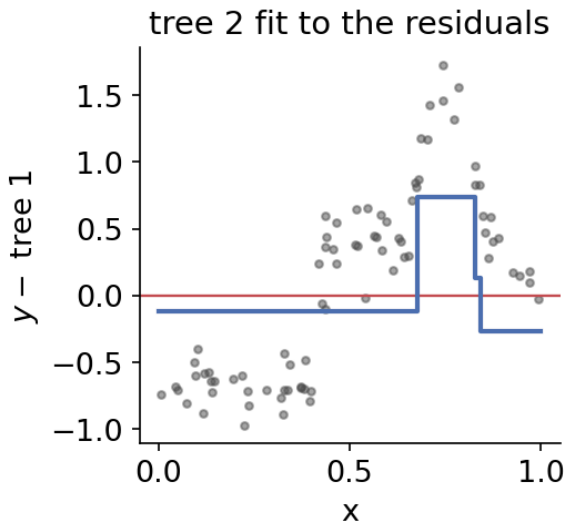
A tree predicts the leaf mean

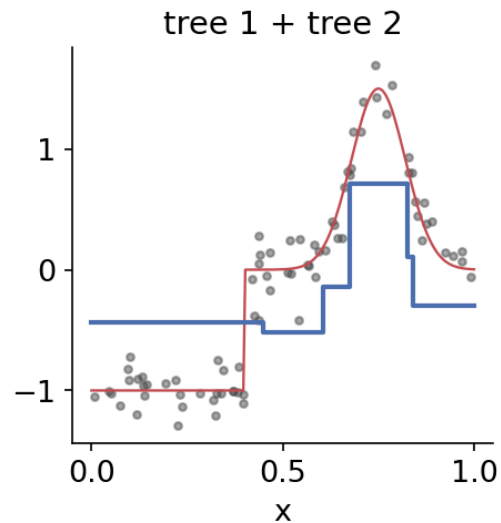


Sum of trees

Fit the next tree to the residuals

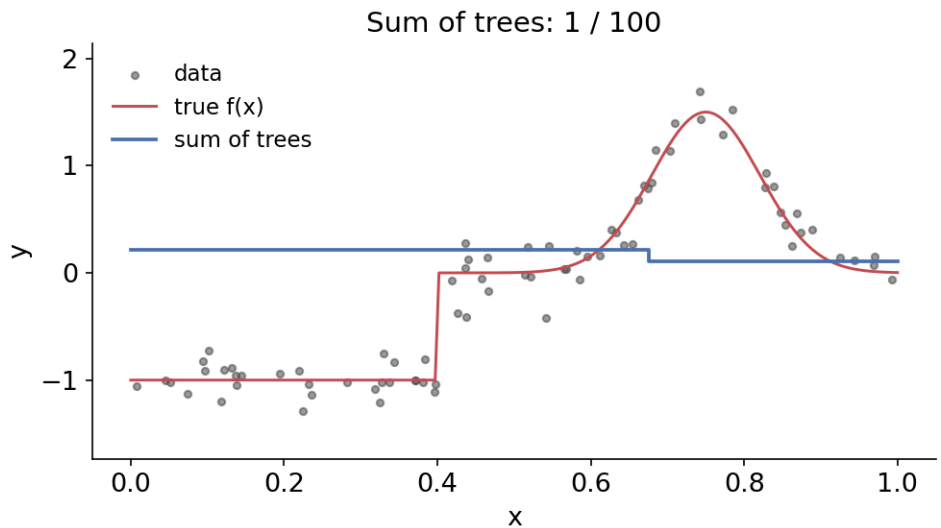






BART is additive

$$f(x) = \sum_{j=1}^{m} g(x; T_j, M_j)$$



Priors instead of cross-validation

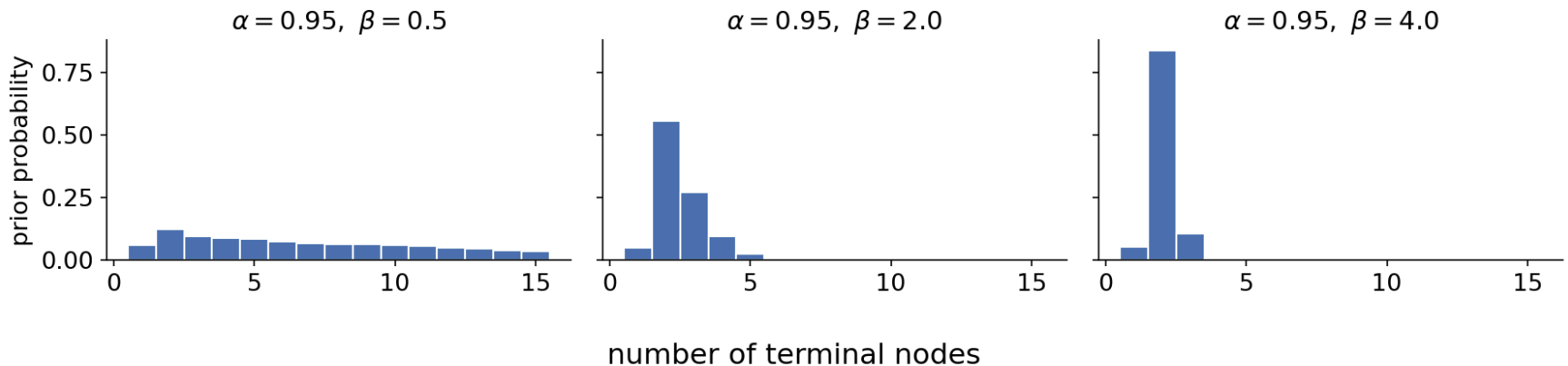
TREE SHAPE	Boosting caps depth by CV — BART puts a **prior on depth**.
LEAF VALUES	Boosting tunes a learning rate — BART **shrinks leaves** toward zero.
NOISE	BART anchors σ^2 **just below the OLS residual variance**.

Prior on tree shape

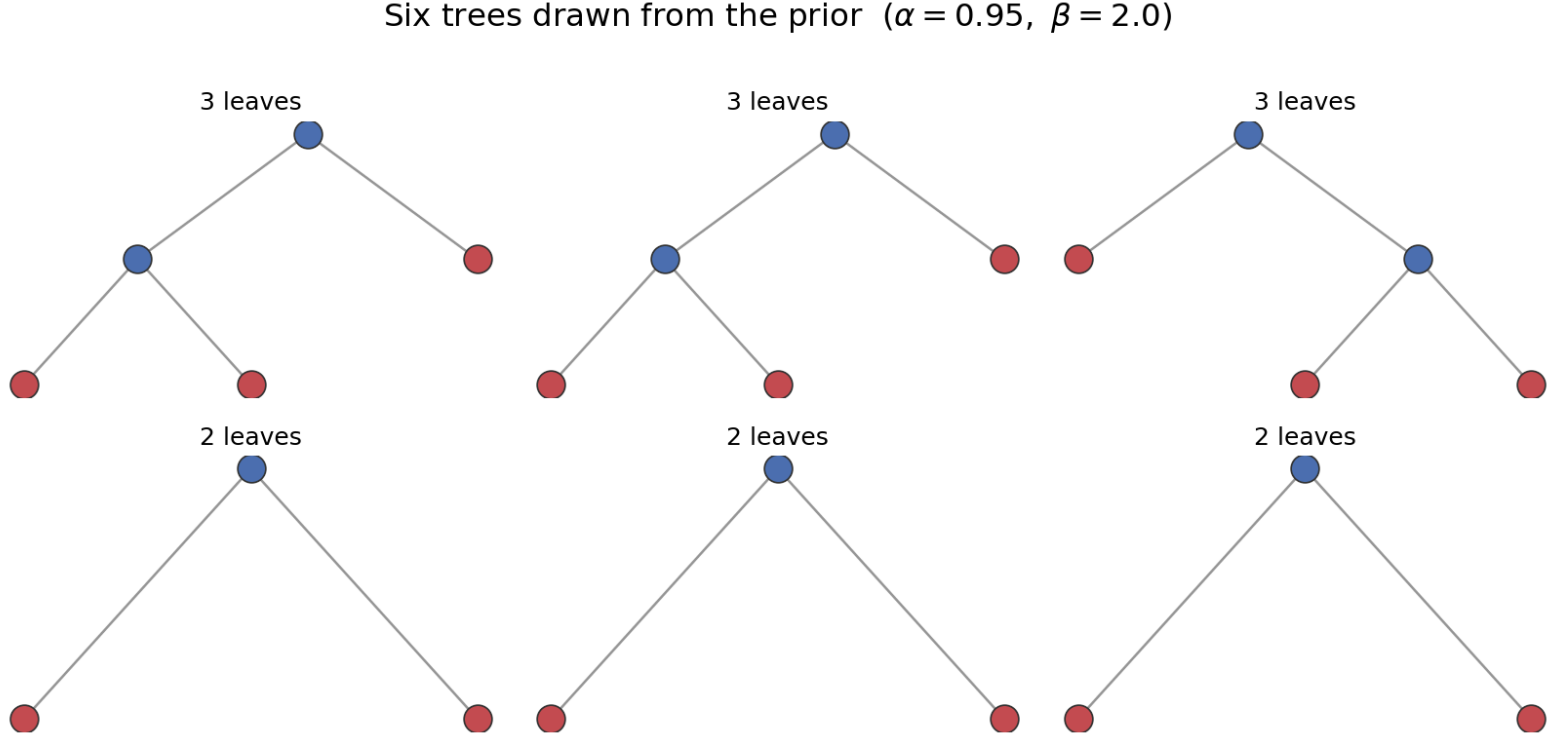
Depth is penalised exponentially

Each split decision at depth d is Bernoulli with $P(\text{split}) = \alpha\,(1+d)^{-\beta}$ (Chipman et al. 1998)

Higher β punishes depth harder → smaller trees



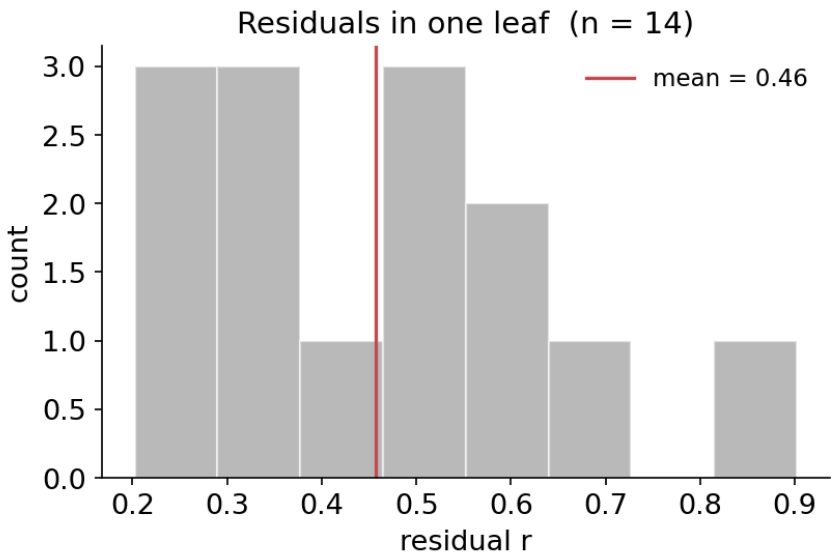
Trees drawn from the prior

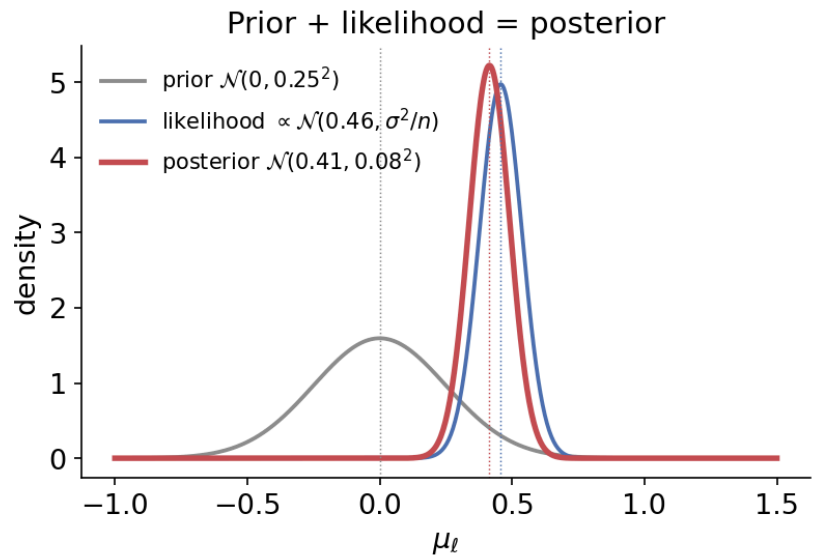


Conjugate leaf updates

Leaf values: a shrinkage estimator

$$\mu_\ell \mid r \sim \mathcal{N}\left(\frac{n_\ell \sigma_\mu^2}{\sigma^2 + n_\ell \sigma_\mu^2}\, \bar{r}_\ell,\ \frac{\sigma^2 \sigma_\mu^2}{\sigma^2 + n_\ell \sigma_\mu^2} \right)$$





One leaf draw

```
def draw_leaf_values(tree, X, r, rng, sigma2, sigma_mu2):
    for lf in tree.leaves():
        n_l = (leaf_of == lf).sum()
        s_l = r[leaf_of == lf].sum()
        post_var  = sigma2 * sigma_mu2 / (sigma2 + n_l * sigma_mu2)
        post_mean = post_var * s_l / sigma2
        tree.mu[lf] = rng.normal(post_mean, np.sqrt(post_var))
```

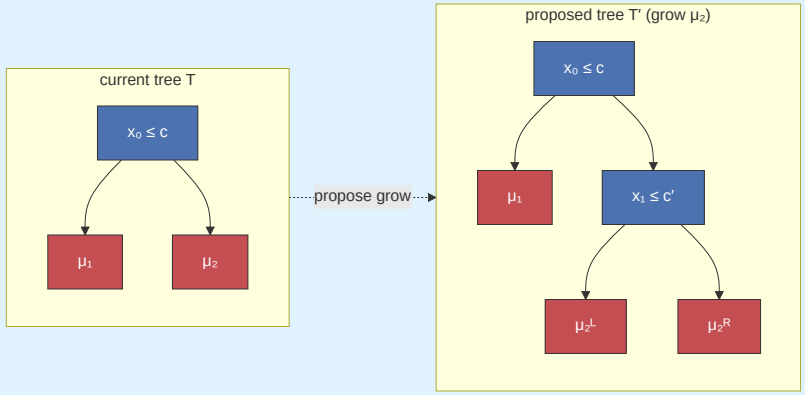
Called once per tree per sweep, after the structure move is accepted.

Metropolis–Hastings on tree structure

Grow and prune

Grow Pick a leaf; split it on a random variable and cut from the data inside it.

Prune Pick a *singly-internal* node; collapse it back to a leaf.



Leaf values integrate out via the marginal likelihood:

$$p(r \mid T, \sigma^2) = \prod_{\ell} \sqrt{\frac{\sigma^2}{\sigma^2 + n_\ell \sigma_\mu^2}} \exp\left(\frac{\sigma_\mu^2 s_\ell^2}{2\sigma^2(\sigma^2 + n_\ell \sigma_\mu^2)} \right)$$

The acceptance ratio

$$\log A = \underbrace{\log \frac{p(r \mid T')}{p(r \mid T)}}_{\text{likelihood}} + \underbrace{\log \frac{\pi(T')}{\pi(T)}}_{\text{prior shape}} + \underbrace{\log \frac{q(T \mid T')}{q(T' \mid T)}}_{\text{move}}$$

Accept T' with probability $\min(1,\, e^{\log A})$. **Prune is the time-reversal of grow.**

The σ update

Noise variance: inverse-χ^2

$$\sigma^2 \mid r \sim \text{Inv-}\chi^2\left(\nu + n,\ \nu\lambda + \textstyle\sum_i r_i^2\right)$$

```python
def draw_sigma2(residuals, nu, lam, rng):
    shape = nu + residuals.size
    scale = nu * lam + np.sum(residuals**2)
    return scale / rng.chisquare(shape)
```

The prior scale λ is calibrated by OLS — set $P(\sigma < \hat{\sigma}) = q$ (default $q = 0.9$).

Putting it together

Blocked Gibbs

one BART sweep

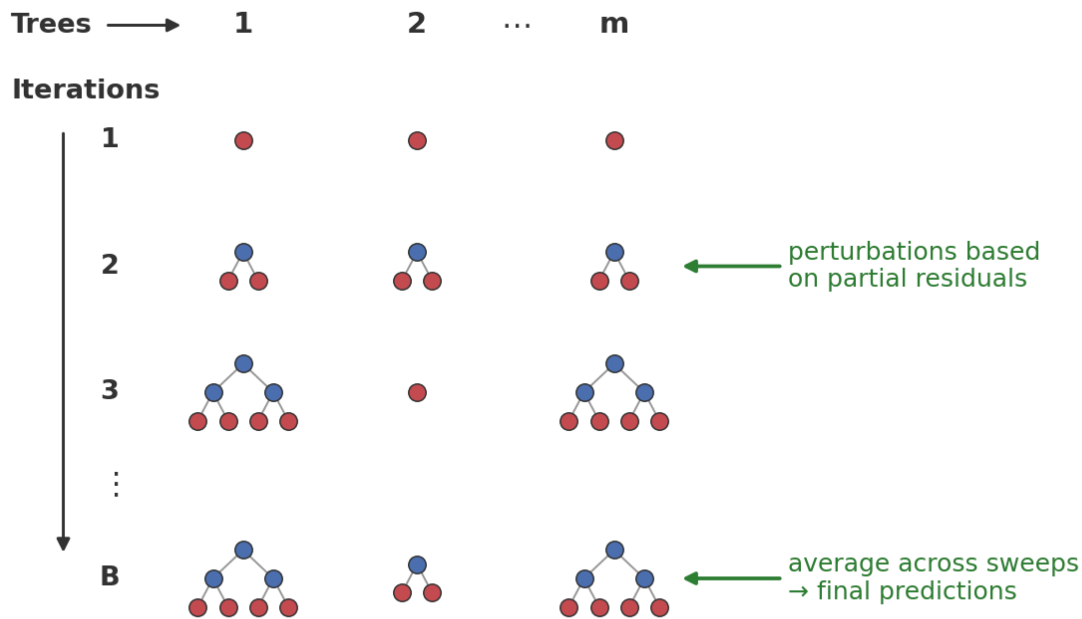
1. **Backfit** — isolate tree j's residual signal

$$R_j = y - \sum_{i \neq j} g(x; T_i, M_i)$$

2. **Structure** — MH grow/prune move for T_j
3. **Leaf values** — conjugate draw of M_j
4. **Noise** — inverse-χ^2 draw of σ^2

A sweep is **Gibbs sampling** over these four blocks.

The BART algorithm — the idea

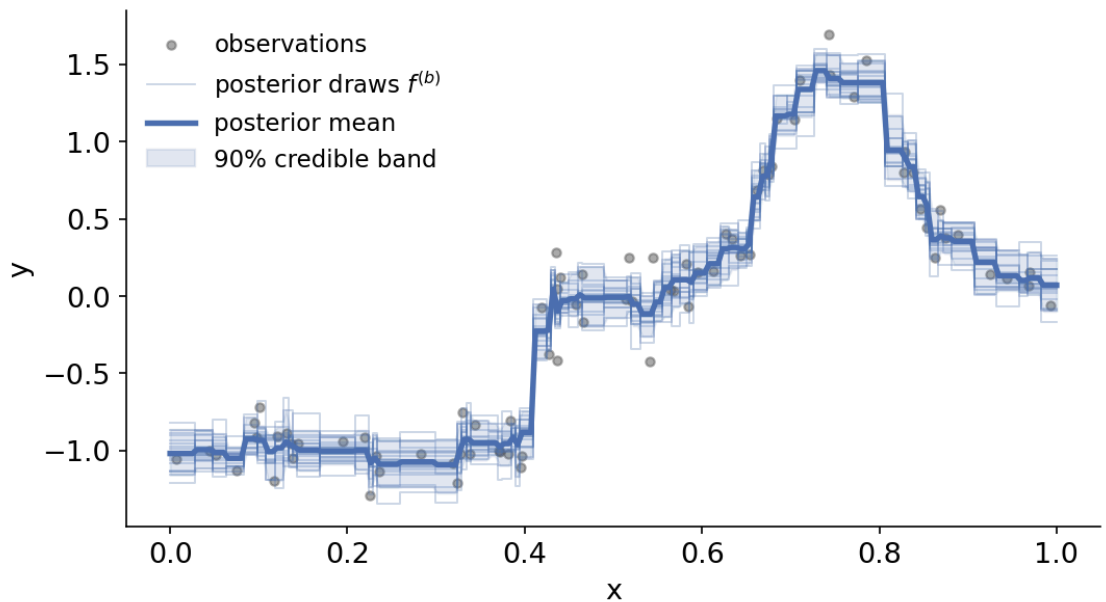


The sampler loop

```
for it in range(n_iter):
    for j in range(m):
        Rj = y - tree_preds.sum(0) + tree_preds[j]        # backfit
        move = "grow" if rng.random() < 0.5 else "prune"
        t_new, logA = propose(move, trees[j], X, Rj, ...)  # MH structure
        if t_new is not None and np.log(rng.random()) < logA:
            trees[j] = t_new
        trees[j] = draw_leaf_values(trees[j], X, Rj, ...)  # leaf
        tree_preds[j] = predict(trees[j], X)
    sigma2 = draw_sigma2(y - tree_preds.sum(0), nu, lam, rng)  # noise
```

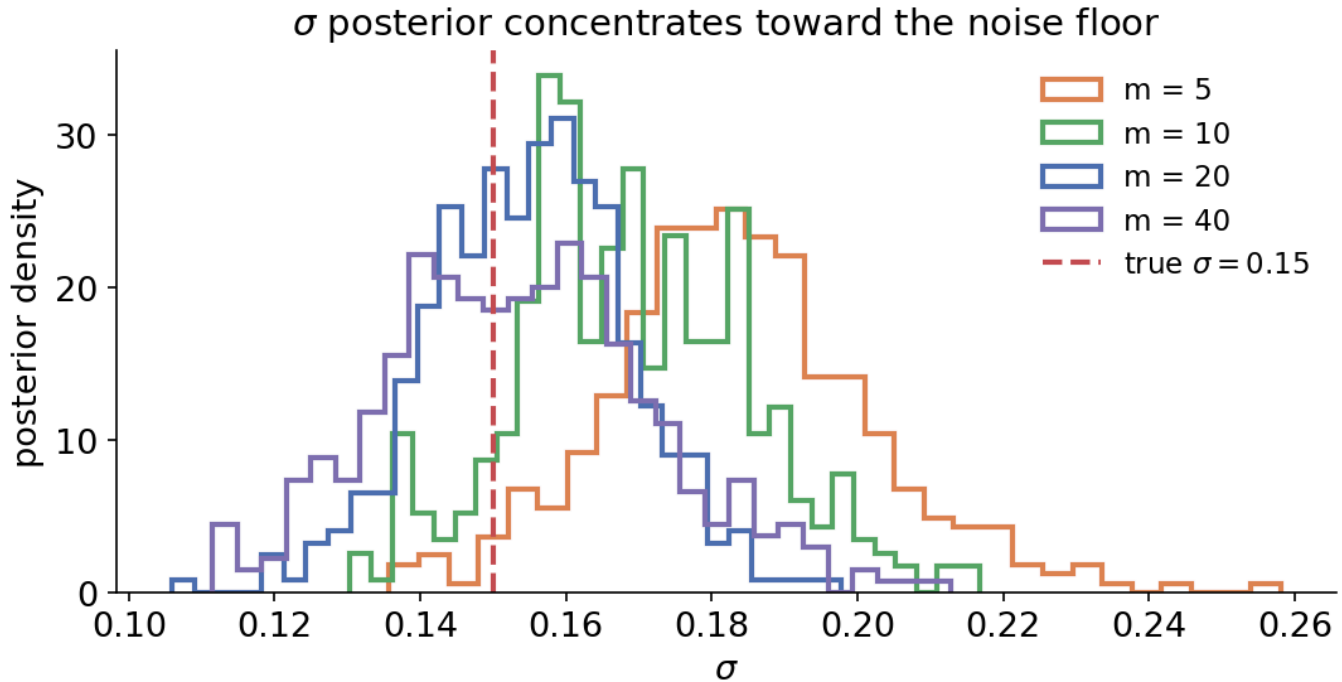
What BART delivers

$$\hat{f}(x) = \frac{1}{B - L} \sum_{b=L+1}^{B} \underbrace{f^{(b)}(x)}_{\text{one draw per sweep}}$$

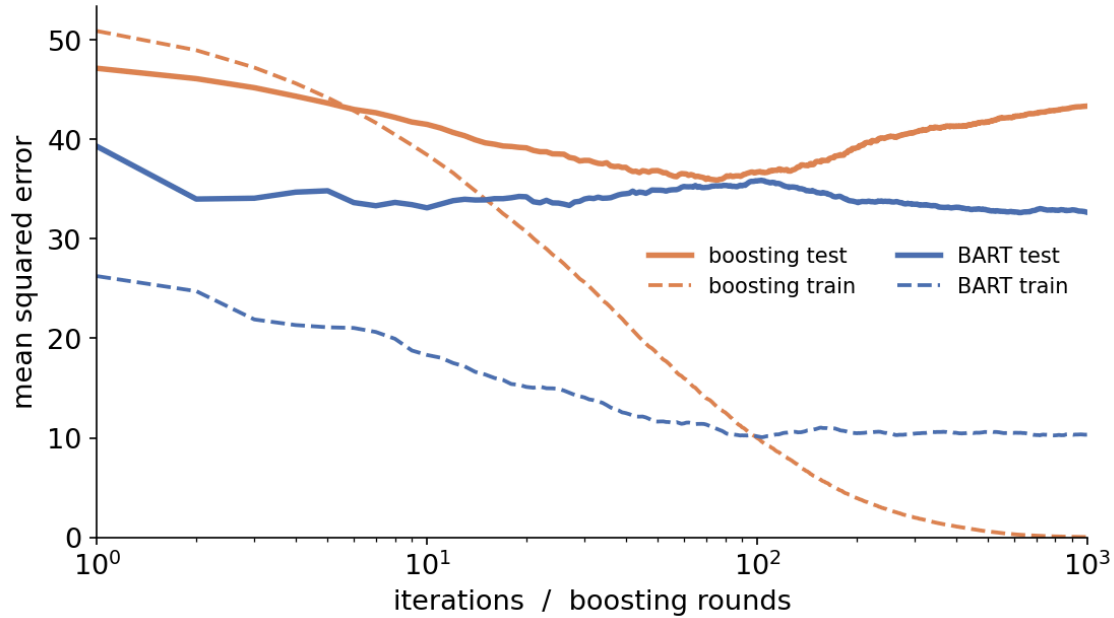


Choosing m

Too few trees inflate $\hat{\sigma}$



More iterations don’t overfit



Let’s run some code!